Fake News Detection Using NLP

A Machine Learning Mini Project | Python, Scikit-learn, TF-IDF, Streamlit

1. Introduction

News spreads very fast on social media, and false or misleading stories spread along with it. Checking every story by hand is slow. Natural Language Processing (NLP) and Machine Learning let a computer learn patterns from examples of real and fake news and give a quick first judgement on a new text. This project builds a small, working system that does exactly that and shows the result in a simple web page.

2. Problem Statement

Given a news headline or article written in English, automatically classify it as **REAL** or **FAKE** using only its text, and present the result through an easy-to-use web interface.

3. Objectives

- Clean raw news text using NLP preprocessing techniques.
- Convert text into numerical features using TF-IDF.
- Train a Logistic Regression classifier and evaluate it with Accuracy, Precision, Recall, F1 score and a Confusion Matrix.
- Build a Streamlit application where a user can paste news and get a prediction.
- Run smoothly on a normal college laptop with beginner-friendly code.

4. Technologies Used

Technology	Purpose
Python 3	Main programming language
Pandas, NumPy	Reading the dataset and handling data/arrays
Regular expressions (re)	Text cleaning and tokenization
Scikit-learn	TF-IDF vectorizer, Logistic Regression, evaluation metrics
Joblib	Saving and loading the trained model
Matplotlib	Confusion matrix and diagram images
Streamlit	Web user interface

5. Dataset Used

The project uses a **small synthetic sample dataset** (500 items: 250 REAL-style and 250 FAKE-style) created by the included script so that the project runs without any download. REAL-style items are written in a neutral reporting tone and FAKE-style items in a sensational, clickbait tone. It is **not real-world news**. The code also accepts any real CSV dataset with *text* and *label* columns.

6. What is NLP?

Natural Language Processing (NLP) is the branch of Artificial Intelligence that helps computers read, process and understand human language. Computers only understand numbers, so NLP converts text into a numerical form that Machine Learning algorithms can use.

7. How the Project Works (Step by Step)

- **Step 1 - Load data:** read *news.csv* with Pandas, check columns, remove empty rows and duplicates.
- **Step 2 - Preprocess:** clean every text (explained below).
- **Step 3 - Split:** 80% training (400 rows) and 20% testing (100 rows), keeping both classes balanced.
- **Step 4 - Vectorize:** convert text to TF-IDF numbers (vocabulary learned from training data only).
- **Step 5 - Train:** fit a Logistic Regression model on the training numbers.
- **Step 6 - Evaluate:** predict on the unseen test set and compute the metrics.
- **Step 7 - Save and use:** save the vectorizer and model with Joblib; the Streamlit app loads them to predict new text.

8. Text Preprocessing

Step	What it does	Example
Lowercase	Makes 'NEWS' and 'news' the same word	BREAKING -> breaking
Remove noise	Removes URLs, HTML tags, punctuation, digits	100%! -> (removed)
Tokenization	Splits the sentence into words	'hidden truth' -> [hidden, truth]
Stopword removal	Removes common words with little meaning	the, is, and, of -> removed

Example from the project: *'BREAKING!!! Visit http://fake.site NOW - The government has 100% hidden the truth!'* becomes *'breaking visit government hidden truth'*.

9. TF-IDF Vectorization

TF-IDF (Term Frequency - Inverse Document Frequency) gives every word a weight. **TF** measures how often a word appears in a text; **IDF** lowers the weight of words that appear in almost every text and raises the weight of rarer, more informative words. Here single words and word pairs (unigrams + bigrams) are used, up to 5000 features.

10. Machine Learning Algorithm: Logistic Regression

Logistic Regression is a simple, fast and easy-to-explain classification algorithm. It learns a weight for each TF-IDF feature, adds them up, and converts the sum into a probability between 0 and 1 (using the sigmoid function). If the probability of REAL is higher than that of FAKE, the text is labelled REAL; otherwise FAKE. It runs in seconds on a normal laptop.

11. Model Training and Prediction

During training the model adjusts its weights to reduce mistakes on the training texts. During prediction, the app applies the **same** preprocessing and the **same** saved TF-IDF vocabulary to the user's text, passes the numbers to the model, and shows the label and the model's probability as a *model score* (not a verified confidence).

12. System Architecture

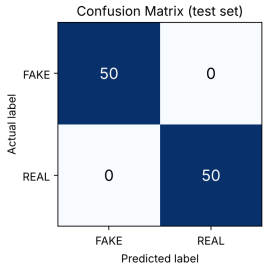


13. Model Evaluation Metrics and Results

The metrics were measured by running *train_model.py* on the 20% test split (100 items never seen during training). **Accuracy** = correct predictions / all predictions. **Precision** = of the texts predicted REAL, how many were really REAL. **Recall** = of the really REAL texts, how many were found. **F1** = balance of precision and recall. The **confusion matrix** counts correct and wrong predictions per class.

Metric (positive class = REAL)	Measured value
Accuracy	100.00%
Precision	100.00%
Recall	100.00%
F1 score	100.00%
5-fold CV accuracy (training part)	100.00% (+/- 0.00%)
Brier score (probability quality, lower is better)	0.030

	Predicted FAKE	Predicted REAL
Actual FAKE	50	0
Actual REAL	0	50



Important note: these scores are perfect because the sample dataset is small and synthetic and the test items share phrasing with the training items; they do **not** show real-world accuracy. On six hand-written articles not in the dataset the model labelled all 6 as expected, but a false claim written in calm, official language was wrongly labelled REAL, showing that the model reads writing style, not facts. Results on a real dataset must be measured separately.

14. Advantages and Limitations

- **Advantages:** simple, fast, runs offline on a normal laptop, easy to explain, easy to retrain with new data, friendly web interface with error handling.
- **Limitations:** learns words and style rather than facts; cannot verify claims or sources; sample dataset is synthetic; English only; the model score is not a calibrated confidence; needs retraining as topics change.

15. Future Scope

Train on a large real-world dataset; try other models (Passive Aggressive Classifier, SVM) and compare; use deep learning models such as BERT; add support for Hindi and other languages; add fact-checking and source credibility checks; calibrate the probabilities; deploy the app online.

16. Conclusion

The project shows the full machine learning workflow for text: preprocessing, TF-IDF, training, evaluation, saving the model and building a web app. It works as a learning demonstration; for real use it must be trained on real news data and combined with human fact-checking.